

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES

Béni Mellal

RAPPORT DE TRAVAUX PRATIQUES

Git & Node.js

Gestion de versions, branches, fusion et collaboration

Module	Génie Logiciel
Encadrant	Pr. BE. ELBAGHAZAOUI
Réalisé par	Fatima Ezzahra IGHIL
Filière	Transformation Digitale Industrielle (TDI)
Niveau	2ème année cycle ingénieur

1. Introduction

Ce TP a pour objectif de prendre en main Git, l'outil de gestion de versions le plus utilisé dans l'industrie, en le combinant avec Node.js pour travailler sur un projet concret. Les compétences travaillées couvrent l'ensemble du cycle de vie d'un projet versionné : initialisation, gestion des branches, fusion, résolution de conflits, rebase, cherry-pick et collaboration via GitHub.

Étape	Description
Étape 1	Initialisation du projet Node.js et premier commit
Étape 2	Gestion des branches – création de la branche dev
Étape 3	Fusion (merge) de dev dans main
Étape 4	Résolution de conflits entre branches
Étape 5	Rebase pour un historique linéaire
Étape 6	Cherry-pick et collaboration GitHub

2. Étape 1 – Initialisation du Projet

2.1 Création du dossier et initialisation Git

La première étape consiste à créer le répertoire du projet, à initialiser le dépôt Git local et à créer le projet Node.js avec npm :

```
$ mkdir mon-projet-node  
$ cd mon-projet-node  
$ git init  
$ npm init -y
```

Initialisation du dépôt Git et du projet Node.js

La commande `git init` crée un dossier caché `.git` qui contiendra tout l'historique des versions. La commande `npm init -y` génère automatiquement un fichier `package.json` avec les valeurs par défaut.

2.2 Ajout du README et du .gitignore

Un fichier `README.md` est créé pour documenter le projet. Le fichier `.gitignore` permet d'exclure le dossier `node_modules/` du suivi Git, évitant ainsi d'alourdir le dépôt :

```
$ echo "# Mon Projet Node" > README.md  
$ echo "node_modules/" >> .gitignore
```

Création du README et configuration du .gitignore

2.3 Création de index.js et premier commit

Un fichier `index.js` minimal est créé, puis tous les fichiers sont ajoutés à l'index Git avant d'effectuer le premier commit :

```
$ echo "console.log('Hello Node');" > index.js  
$ git add README.md .gitignore index.js package.json  
$ git commit -m "Initialisation du projet Node (README, .gitignore, index.js)"
```

Premier commit – snapshot initial du projet

3. Étape 2 – Gestion des Branches

3.1 Création de la branche dev

Les branches permettent de travailler sur une fonctionnalité sans affecter la branche principale. La branche `dev` est créée et activée :

```
$ git branch dev  
$ git checkout dev
```

Création et activation de la branche dev

3.2 Ajout de la fonction addition

Sur la branche `dev`, le fichier `index.js` est modifié pour y ajouter une fonction `addition`, avec export du module pour une utilisation potentielle en tant que bibliothèque :

```
// index.js (branche dev)  
function addition(a, b) {  
  return a + b;  
}  
if (require.main === module) {  
  console.log("Résultat:", addition(5, 3));  
}  
module.exports = { addition };
```

```
$ git add index.js
$ git commit -m "Ajout de la fonction addition dans index.js"
Commit sur la branche dev
```

4. Étape 3 – Fusion (Merge)

Une fois le développement sur la branche dev terminé, on retourne sur la branche main et on fusionne les modifications :

```
$ git checkout main
$ git merge dev
Fusion de la branche dev dans main
```

Git réalise ici un fast-forward merge : comme main n'a pas évolué depuis la création de dev, Git se contente d'avancer le pointeur de main jusqu'au dernier commit de dev. Aucun conflit n'est généré.

5. Étape 4 – Résolution de Conflits

5.1 Création de la branche feature

Une branche feature est créée à partir de main pour y ajouter une fonction soustraction :

```
$ git checkout -b feature

// index.js (branche feature)
function addition(a, b) { return a + b; }
function soustraction(a, b) { return a - b; }
if (require.main === module) {
  console.log("Résultat addition:", addition(5, 3));
  console.log("Résultat soustraction:", soustraction(5, 3));
}
module.exports = { addition, soustraction };

$ git commit -am "Ajout de la fonction soustraction"
Ajout de soustraction sur la branche feature
```

5.2 Modification concurrente sur main

En parallèle, des commentaires de documentation sont ajoutés sur la fonction addition directement sur la branche main :

```
$ git checkout main

// addition(a, b) -> retourne la somme de a et b
function addition(a, b) {
  return a + b; // simple addition
}

$ git commit -am "Doc: améliorer les commentaires de addition"
```

Commit de documentation sur main

5.3 Détection et résolution du conflit

La tentative de fusion de feature dans main génère un conflit car les deux branches ont modifié les mêmes lignes de index.js :

```
$ git merge feature
CONFLICT (content): Merge conflict in index.js
Automatic merge failed; fix conflicts and then commit the result.
```

Message de conflit détecté par Git

Git insère des marqueurs de conflit dans le fichier. Le fichier doit être édité manuellement pour combiner les deux versions :

```
<<<<<< HEAD
// addition(a, b) -> retourne la somme de a et b
function addition(a, b) {
  return a + b; // simple addition
}
=====
function addition(a, b) { return a + b; }
function soustraction(a, b) { return a - b; }
>>>>>> feature
```

Marqueurs de conflit dans index.js

La résolution consiste à combiner les deux apports : conserver les commentaires de documentation ET la fonction soustraction :

```
// addition(a, b) -> retourne la somme de a et b
function addition(a, b) {
  return a + b; // simple addition
}
function soustraction(a, b) { return a - b; }
if (require.main === module) {
  console.log("Résultat addition:", addition(5, 3));
  console.log("Résultat soustraction:", soustraction(5, 3));
}
module.exports = { addition, soustraction };
```

Version finale après résolution du conflit

```
$ git add index.js
$ git commit
```

Finalisation du merge après résolution

6. Étape 5 – Rebase

6.1 Création de la branche bugfix

Une branche bugfix est créée pour rendre la fonction addition plus robuste en gérant les valeurs non numériques (NaN) :

```
$ git checkout -b bugfix
```

```
// index.js (bugfix) - rendre addition plus robuste
function addition(a, b) {
  const x = Number(a), y = Number(b);
  if (Number.isNaN(x) || Number.isNaN(y)) return 0;
  return x + y;
}
```

```
$ git commit -am "Bugfix: addition robuste (gère les NaN)"
```

Commit du correctif sur la branche bugfix

6.2 Rebase de main sur bugfix

Le rebase rejoue les commits de main au-dessus de ceux de bugfix, produisant un historique linéaire et plus lisible qu'un merge commit :

```
$ git checkout main
$ git rebase bugfix
```

Rebase de main au-dessus de bugfix

Contrairement au merge qui crée un commit de fusion, le rebase réécrit l'historique pour que les commits apparaissent dans une séquence continue. Cela facilite la lecture du log et le débogage.

7. Étape 6 – Cherry-pick et GitHub

7.1 Cherry-pick

Le cherry-pick permet d'appliquer un commit précis d'une branche sur une autre, sans fusionner toute la branche. On récupère d'abord le hash du commit cible :

```
$ git log --oneline
3f5c2d3 Bugfix: addition robuste (gère les NaN)
1a2b3c4 Ajout de la fonction soustraction
9d8e7f6 Doc: améliorer les commentaires de addition
```

Exemple de log compact

```
$ git checkout feature
$ git cherry-pick 3f5c2d3
```

Application du commit bugfix sur la branche feature

En cas de conflit lors du cherry-pick, il faut résoudre manuellement puis continuer avec `git cherry-pick --continue`.

7.2 Publication sur GitHub

Pour collaborer avec d'autres développeurs, le dépôt local est lié à un dépôt distant GitHub :

```
$ git remote add origin git@github.com:VotreUser/mon-projet-node.git
$ git branch -M main
$ git push -u origin main
```

Association et premier push vers GitHub

La configuration de l'identité Git est nécessaire pour que les commits soient correctement attribués :

```
$ git config --global user.name "Fatima Ezzahra IGHIL"
$ git config --global user.email "fatima@email.com"
```

Configuration de l'identité Git globale

7.3 Bonnes pratiques de collaboration

Pour travailler efficacement en équipe sur GitHub, les pratiques recommandées sont :

- Utiliser des branches par fonctionnalité (feature/nom-fonctionnalite)
- Soumettre des Pull Requests pour toute intégration vers main
- Effectuer des code reviews avant de fusionner
- Activer la protection de branche sur main pour éviter les push directs
- Synchroniser régulièrement avec git pull origin main

8. Synthèse des Commandes Git

Commande	Rôle
<code>git init</code>	Initialiser un dépôt Git local
<code>git add <fichier></code>	Ajouter un fichier à l'index (staging)
<code>git commit -m "msg"</code>	Enregistrer un snapshot avec un message
<code>git branch <nom></code>	Créer une nouvelle branche
<code>git checkout <branche></code>	Changer de branche
<code>git merge <branche></code>	Fusionner une branche dans la branche courante
<code>git rebase <branche></code>	Rejouer les commits pour un historique linéaire
<code>git cherry-pick <hash></code>	Appliquer un commit précis sur la branche courante
<code>git log --oneline</code>	Afficher l'historique compact
<code>git push origin main</code>	Envoyer les commits vers le dépôt distant
<code>git pull origin main</code>	Récupérer et intégrer les modifications distantes

9. Conclusion

Ce TP a permis de couvrir l'ensemble des fonctionnalités essentielles de Git dans un contexte de développement Node.js réaliste. Les notions abordées — branches, merge, résolution de conflits, rebase et cherry-pick — constituent le socle indispensable pour tout développeur travaillant en équipe.

La maîtrise de Git est particulièrement stratégique dans un cursus de Transformation Digitale Industrielle, où la gestion rigoureuse du code source, la traçabilité des modifications et la

collaboration multi-développeurs sont des exigences quotidiennes dans les projets industriels.